



Design and Analysis of Algorithms

Chapter 7 – Dynamic Programming

Instructor: Suleiman Gargaare

Dynamic Programming (DP)

- ◆ Dynamic Programming is an algorithm design method that can be used when the solution to a problem can be viewed as the result of a sequence of decisions.
- ◆ For some problems, an optimal sequence of decisions can be found by making the decisions one at a time using greedy method.

Cont..

- ◆ For other problems, it is not possible to make step-wise decisions based on only local information.
- ◆ One way to solve such problems is to try all possible decision sequences. However time and space requirement is prohibitive.
- ◆ DP reduces those possible sequences not leading to optimal decision.

Divide & Conquer vs. Dynamic Programming

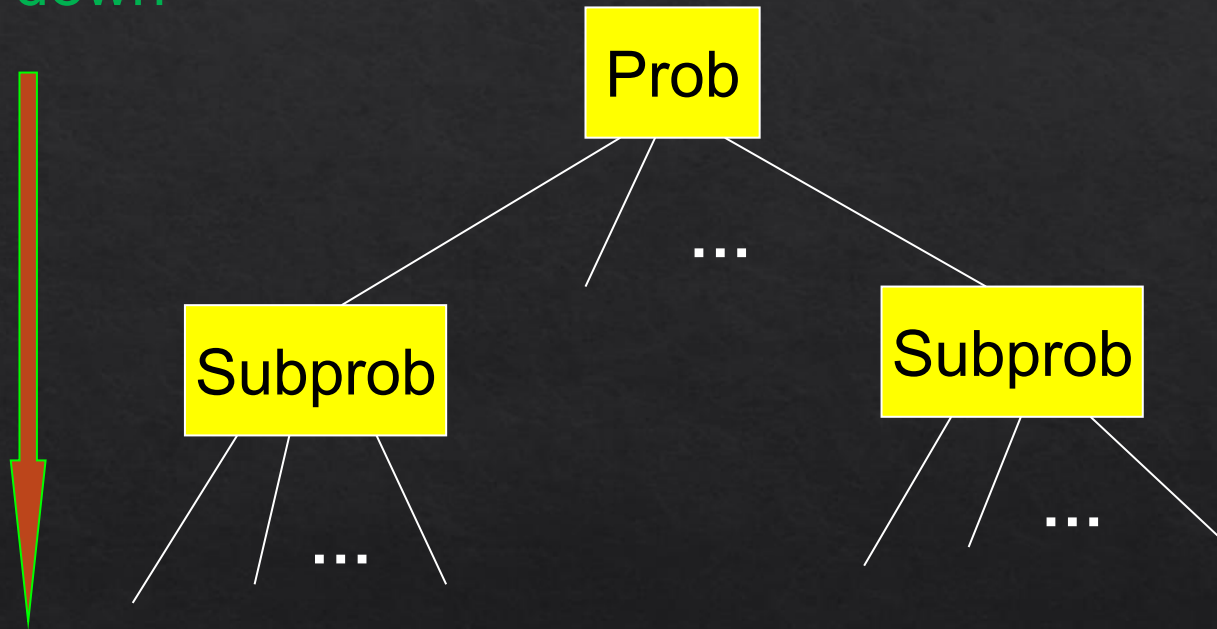
- ◆ Both techniques split their input into parts, find sub-solutions to the parts, and combine solutions to sub-problems.
- ◆ In divide and conquer, solution to one sub-problem may not affect the solutions to other sub-problems of the same problem.
 - ◆ In **dynamic programming**, sub-problems are not **independent**. Sub-problems may share sub-sub-problems

Cont..

- ◇ Dynamic programming, like divide and conquer method, solves problems by combining the solutions to sub-problems.
- ◇ Divide and conquer algorithms:
 - Partition the problem into independent sub-problem.
 - Solve the sub-problem recursively and
 - Combine their solutions to solve the original problem.
- ◇ In contrast, dynamic programming is applicable when the sub-problems are not independent.

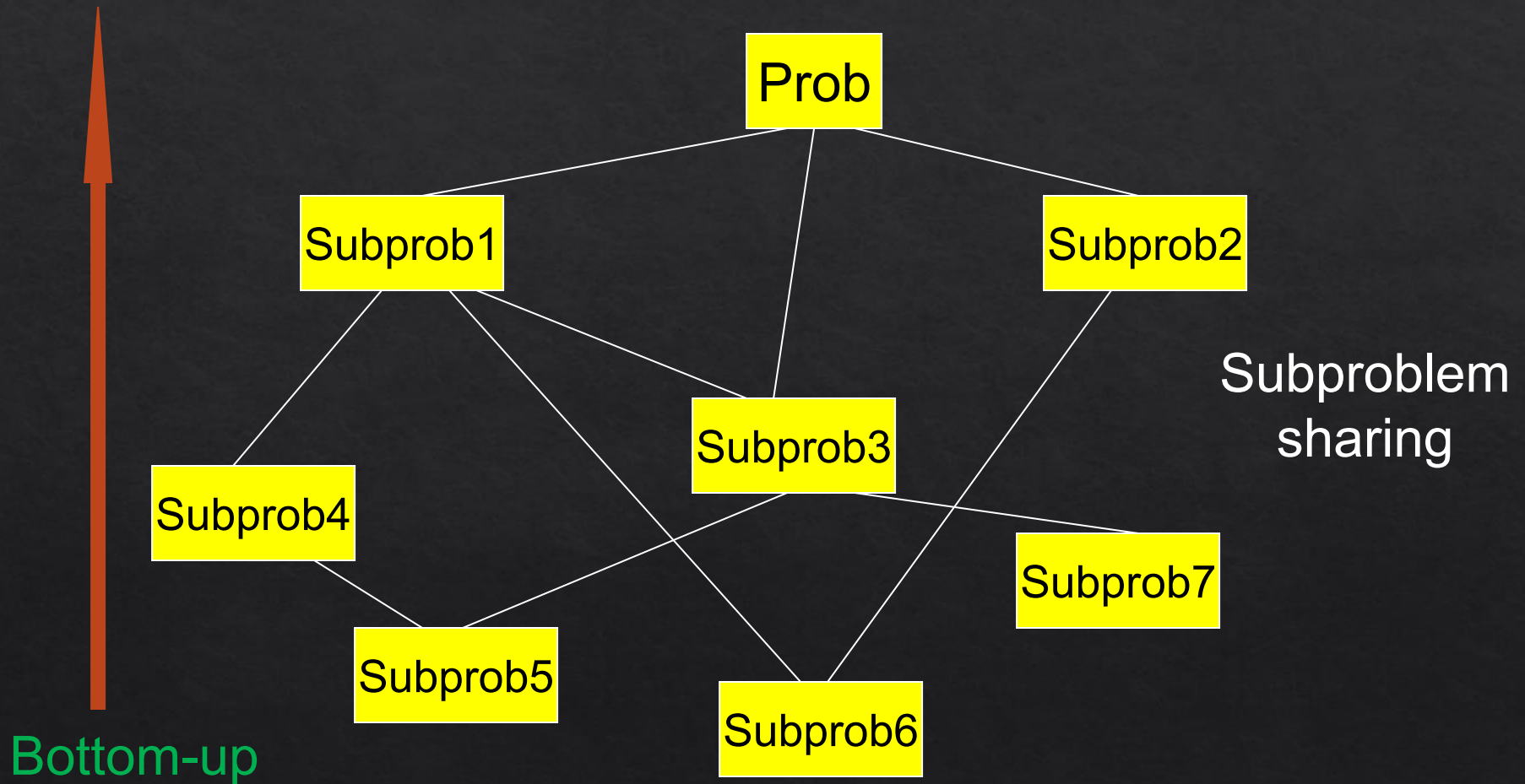
Divide-and-Conquer

Top-down



Independent sub-problems (no overlapping work).

Cont..



Greedy vs. Dynamic Programming

- ◆ Both techniques are an algorithm design technique for **optimization problems** (*minimizing or maximizing*), and both build solutions from a collection of choices of individual elements.
- ◆ The greedy method **computes its solution by making its choices in a serial forward fashion**, never looking back or revising previous choices.
- ◆ Dynamic programming **computes its solution forward/backward** by synthesizing them from smaller sub-solutions, and by trying many possibilities and choices before it arrives at the optimal set of choices.
- ◆ There is no a priori test by which one can tell if the Greedy method will lead to an optimal solution.
- ◆ By contrast, there is a test for Dynamic Programming, called **the Principle of Optimality**

The Principle of Optimality

- ◆ In Dynamic Programming an optimal sequence of decisions is obtained by making explicit appeal to the principle of optimality.
- ◆ **Definition:** A problem is said to satisfy the Principle of Optimality if the sub-solutions of an optimal solution of the problem are themselves optimal solutions for their sub-problems.
- ◆ In solving a problem, we make a sequence of decisions $D_1, D_2, \dots, D_n$. If this sequence is optimal, then the k decisions also be optimal

Cont..

Examples:

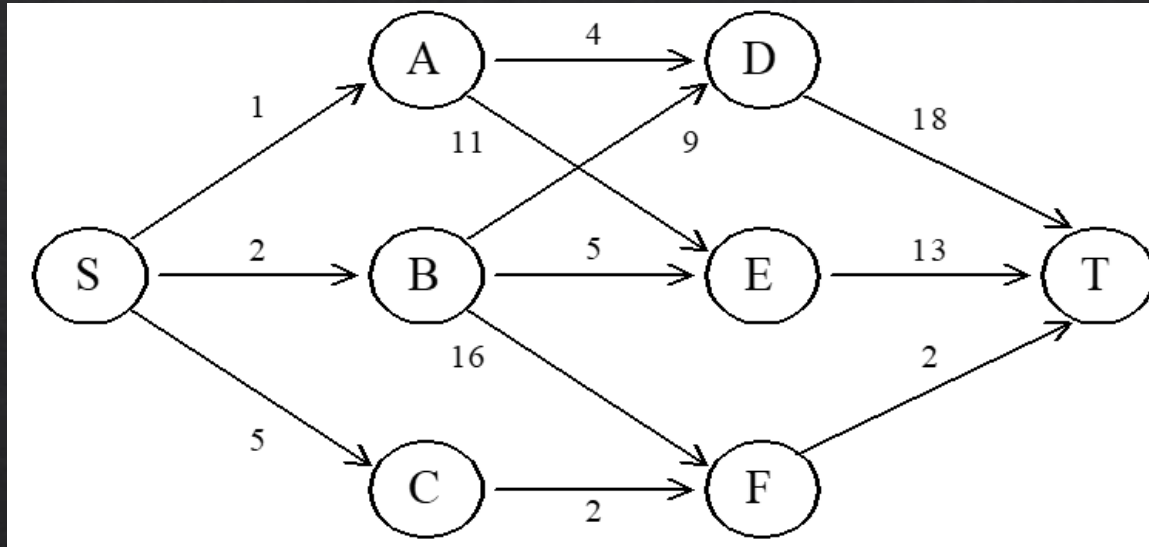
- ◆ The **shortest path problem** satisfies the principle of optimality.
 - ◆ This is because if $a, x_1, x_2, \dots, x_n, b$ is a shortest path from node **a** to node **b** in a graph, then the portion of x_i to x_j on that path is a shortest path from x_i to x_j .
- ◆ Dynamic Programming reduces computation by
 - ◆ Storing solution to a sub-problem the first time it is solved.
 - ◆ Looking up the solution when sub-problem is encountered again.
 - ◆ Solving sub-problems in a bottom-up or top-down fashion.

Dynamic Programming Approaches

- ◆ To solve a problem by using dynamic programming:
 - ◆ Find out the recurrence relations.
 - ◆ Dynamic programming is a technique for efficiently computing recurrences by storing partial results.
 - ◆ Represent the problem by a multistage graph.
 - ◆ In summary, if a problem can be described by a multistage graph, then it can be solved by dynamic programming
- ◆ **Forward approach and backward approach:**
 - ◆ If the recurrence relations are formulated using the backward approach, then the relations are solved beginning with the last decision.
 - ◆ If the recurrence relations are formulated using the forward approach, then the relations are solved starting from the beginning until we reach to the final decision.

The Shortest Path in Multistage Graphs

- Find the shortest path in multistage graphs for the following example?



- ◇ The greedy method can not be applied to this case: (S, A, D, T) $1+4+18 = 23$.
- ◇ The real shortest path is:
(S, C, F, T) **$5+2+2 = 9$** .

The Shortest Path

- ◆ Given a multi-stage graph, how can I find a shortest path?
- ◆ **Forward approach:** Let $p(i,j)$ denote the minimum cost path from vertex j to the terminal vertex T . Let $\text{COST}(i,j)$ denote the cost of $p(i,j)$ path. Then using the forward approach, we obtain:

$$\text{COST}(i,j) = \min \{ \text{COST}(i,j), c(i,k) + \text{COST}(k,j) \}$$

- ◆ **Backward approach:** Let $p(i,j)$ be a minimum cost path from vertex S to a vertex j in V_j . Let $\text{COST}(i,j)$ be the cost of $p(i,j)$.

$$\text{COST}(i,j) = \min \{ \text{COST}(i,j), \text{COST}(i,k) + c(k,j) \}$$

NB. If (i, j) is not element of E then $\text{COST}(i, j) = + \infty$.

Algorithm

```
procedure shortest_path (COST[ ], A[ ],  $n$ )  
  //cost[i,j] is the cost of edges[i,j] and A[i,j] is the shortest path from i to j  
  //cost[i,i] is 0.0  
  for  $i = 1$  to  $n$  do  
    for  $j = 1$  to  $n$  do  
       $A(i, j) := \text{COST}(i, j)$  //copy cost into A  
    for  $k = 1$  to  $n$  do  
      for  $i = 1$  to  $n$  do  
        for  $j = 1$  to  $n$  do  
           $A(i, j) = \min(A(i, j), A(i, k) + A(k, j));$   
        end for  
      end for  
    end for  
  return  $A(1..n, 1..n)$   
end shortest_path
```

This algorithm runs in time $O(n^3)$

THE END

THANK YOU!